

6.7 Querying Graph Databases: Neo4j

Cypher is declarative query language for graphs.

declarative vs imperative query language

`MATCH (b:Beer) RETURN b.name`

GUI or Python

6.8 Querying Graph Databases - Basic

Operations

<u>SQL</u>	<u>Cypher</u>
FROM	MATCH
WHERE	WHERE
SELECT	RETURN

Patterns

- Nodes: `(b:Beer {name: "Bud"})`
- Relationships: `(bar:Bar) -[s:sells] -> (beer:Beer)`
- Patterns can be written continuously or separated by commas.

MATCH-WHERE-RETURN: over a node

- A table of an entity becomes a node in a graph DB.
- Querying over a node: `(var: Label)`

MATCH-WHERE-RETURN: over a Relationship

- A table of a relationship becomes, naturally, a relationship in a graph DB.
- Querying over a ~~node~~^{relationship}: `(...) -[var: label] -> (...)`

Using properties of Nodes and Relationships

- Node: `(var: Label {Properties})`
- Relationship: `(...) -[var: Label {Properties}] -> (...)`

Join queries

- In SQL, you assemble the tables involved using joins.
- In a graph DB, you specify a pattern to traverse the graph.

6.9 Querying Graph Databases - Advanced capabilities

Aggregate

- Similar to SQL
 - By applying aggregate functions over groups.
- No separate "groupBy" clause
 - Simply put grouping attributes in "RETURN" together with aggregates.

e.g. `(l) -[s:sells] -> (l) RETURN MAX(s.price),
AVG(s.price)`

Chaining - Connecting Subqueries

- MATCH-WHERE-RETURN can be extended to a pipeline.
- MATCH-WHERE-(WITH-MATCH-WHERE)*-RETURN.
- WITH clause is the connection from one stage to the next.
 - specifies variables/functions to pass on, and give them aliases to refer to.

• E.g.

... WITH beer.name AS beer, COUNT(*) AS countSales, AVG(s.price) AS avgPrice

MATCH ()-[s:sells]-(l) &

WITH MAX(s.price) AS maxPrice ~~WHERE~~

MATCH ()-[s:sells]→(beer:Beer)

WHERE s.price = maxPrice

MATCH ()-[s:sells]-(beer:Beer { brewer:

"AB InBev"} WITH beer.name AS beer, AVG(s.price) AS avgPrice, COUNT(*) AS countSales WHERE countSales >= 2

RETURN beer, avgPrice